

Fraud Investigator: Code Review and Fix Spec

Repository: https://github.com/r4ghv/Fraud_Invsestigator (branch **main**, commit **023b72e**). Reviewed 23 September 2026.

How to use this document

This is a hand-off spec for a coding model. Each finding has an ID, the exact file and line, what is wrong, why it matters, a proposed fix, and acceptance criteria. Work through Section 6 (Fix order) top to bottom. Line numbers refer to commit 023b72e; re-check them before editing, since earlier fixes shift lines.

Ground rules for the implementing model

- Keep the project's core design: **all decisions (verdict, actions, routes, SAR) are deterministic code**. An LLM may only rephrase explanations. Do not add LLM calls to decision paths.
- Every numeric threshold must come from `fraud_agent/policies/policy.yaml`, not from literals in code.
- The authoritative task spec is `fraud_agent/data/README.md` (Fraud Policy R1 to R10, Section 3a, Answer Format). When this document and that README disagree, follow the README.
- Write a failing test first for each bug fix (IDs C1 to H9), then fix it.
- Do not change the answer JSON schema; only change the values the agent produces.
- `fraud_agent/data/transactions.csv` and `identity.csv` are git-ignored (about 708 MB). Tests must use small synthetic rows so they run without them.

1. What the repo contains

Two unrelated projects share one repository and no code:

Project	Location	What it does
A. Face ID + Blockchain	README.md (root), <code>src/</code> , <code>scripts/</code> , <code>contracts/</code> , <code>models/</code>	Photo -> YuNet face detection -> SFace embedding -> Bing reverse image search via Puppeteer -> pick a social media link -> write attestation to an EVM contract.
B. Fraud agent	<code>fraud_agent/</code>	TigerGraph HHGOA task. Reads <code>case_pack.csv</code> (20 cases), loads IEEE-CIS transactions, runs 5 pattern detectors plus policy rules, writes 20 answer JSON files to <code>fraud_agent/cases/</code> .

2. Summary of the current state

- **Core idea is sound:** rules decide, AI only explains. This is correct for regulated fraud work.
- **Output is skewed:** of 20 committed answers, 10 are fraud, 10 uncertain, **0 legitimate**. The README states about half the cases are legitimate and that an agent that blocks everything scores badly.
- **12 of 20** answers carry the pattern `card_not_present_new_device`, caused by bug C1.
- **5 of 11** modules in `fraud_agent/src` are unused; one (`next_best_action.py`) would crash if called.
- **Tests:** 4, all on `policy_engine.py`. None on detectors, guards, or the answer format.
- **Face pipeline:** never compares the found face with the input face, yet writes a permanent on-chain "match".

Observed outputs (from `fraud_agent/cases/*.json`)

Case	Verdict	Pattern	p	Exposure	SAR	tool_calls
HHG-001	uncertain	none	0.67	0	no	3
HHG-002	uncertain	<code>cnp_new_device</code>	0.69	0	no	6

Case	Verdict	Pattern	p	Exposure	SAR	tool_calls
HHG-003	fraud	cnp_new_device	0.90	300.01	no	9
HHG-004	fraud	cnp_new_device	0.90	235.47	yes	12
HHG-005	uncertain	none	0.30	0	no	15
HHG-006	fraud	cnp_new_device	0.90	1906.07	yes	18
HHG-007	uncertain	cnp_new_device	0.79	0	no	21
HHG-008	fraud	cnp_new_device	0.90	335.71	yes	24
HHG-009	fraud	none	0.90	30.02	yes	27
HHG-010	uncertain	account_takeover	0.79	0	yes	30
HHG-011	fraud	card_testing	0.90	32.68	yes	33
HHG-012	uncertain	cnp_new_device	0.69	0	no	36
HHG-013	uncertain	none	0.37	0	no	39
HHG-014	uncertain	none	0.29	0	no	42
HHG-015	uncertain	account_takeover	0.76	0	yes	45
HHG-016	fraud	cnp_new_device	0.90	209.04	yes	48
HHG-017	uncertain	none	0.31	0	no	51
HHG-018	fraud	cnp_new_device	0.90	529.94	no	54
HHG-019	uncertain	cnp_new_device	0.79	0	yes	57
HHG-020	uncertain	none	0.30	0	no	60

cnp_new_device = card_not_present_new_device. Note: HHG-010 and HHG-019 file a SAR while verdict is uncertain and case exposure is 0 (see H6). tool_calls grows by 3 per case (see M12).

3. Current flow of the fraud agent (fraud_agent/src/agent.py, investigate())

```
case_pack row
-> Store.card_window(): +/- 60 day window (includes txns AFTER the alert)
-> 5 detectors scan the whole window (not anchored on flagged txn)
-> pick first hit in fixed order (cnp_new_device before cnp_burst)
-> trip guard / R7 reset `pattern` only (signals list `sigs` unchanged)
-> probab_from(sigs, ...) (scores dropped signals)
-> ~15 if/elif branches, hardcoded numbers
-> answer JSON
```

4. Findings

Severity: CRITICAL = wrong answers now; HIGH = policy mismatch or likely wrong answers; MEDIUM = correctness risk, dead code, performance; LOW = hygiene. "Verified" means I reproduced it with synthetic data against the committed code.

[CRITICAL] C1. cnp_new_device fires with zero device evidence

fraud_agent/src/pattern_detectors.py:45

WHAT IS WRONG

- The detector should fire only when id_15 == 'New' or id_23 is a proxy value. When neither holds, its last line still returns True with cnp_burst's confidence.
- It runs before cnp_burst in agent.py's `order` list, so it swallows every CNP hit. The label card_not_present_fraud can never be produced.

WHY IT MATTERS

12/20 answers carry this label. Pattern is a scored field.

VERIFIED

Verified: 4 online txns with an empty identity dict -> (True, 0.65, ['3', '4']).

FIX

```
# end of cnp_new_device()
if not (new or proxy):
    return False, 0.1, []
return True, (0.8 if (new and proxy) else 0.75), ids
```

ACCEPTANCE CRITERIA

- With empty identity data, cnp_new_device returns hit=False.
- A CNP burst without new device yields pattern card_not_present_fraud.

[CRITICAL] C2. Dropped signals still drive fraud probability

fraud_agent/src/agent.py:45, 51-62, 16-25

WHAT IS WRONG

- `sigs` is built on line 45. The R7 check (line 51) and trip guard (line 58) later set pattern='none', but `sigs` is never updated.
- prob_from() then uses the max confidence of the signal that was just discarded.

WHY IT MATTERS

HHG-001: pattern none, p=0.67, escalated. Without the dropped signal p would be 0.32.

VERIFIED

Verified: prob_from([(True,0.7)], 0.61, False, False) = 0.67; prob_from([], 0.61, False, False) = 0.32. Matches HHG-001.

FIX

```
dropped = set()
if r7: dropped |= {name for _, _, name, _ in order}
if trip_detected: dropped.add("out_of_region_use")
sigs = [(h, c) for h, c, name, _ in order if h and name not in dropped]
```

ACCEPTANCE CRITERIA

- Unit test: when trip guard fires, prob_from receives no out_of_region signal.
- Re-running HHG-001 no longer gives p=0.67 from a dropped signal.

[CRITICAL] C3. No case is ever judged legitimate; R7 almost never fires

fraud_agent/src/agent.py:51-57, 101

WHAT IS WRONG

- Every customer_report case becomes fraud with p=0.9 unless R7 fires.
- R7 compares the flagged amount with the median of ALL txns in the window, not the same merchant/product. It never checks a monthly rhythm, which R7 requires ("same merchant, same amount, monthly").
- All remaining cases default to uncertain.

WHY IT MATTERS

README: "Half the cases are legitimate. Many look suspicious. An agent that blocks everything scores badly."

FIX

```
same = [r for r in window
        if r['ProductCD'] == flag['ProductCD']
        and r['TransactionID'] != flag['TransactionID']]
med_same = median(float(r['TransactionAmt']) for r in same) if same else 0
gaps = day_gaps_sorted(same)          # days between consecutive same-product txns
monthly = len(gaps) >= 2 and all(25 <= g <= 35 for g in gaps)
r7 = (len(same) >= 3 and monthly and med_same > 0
      and abs(amt - med_same) / med_same < 0.10)
```

ACCEPTANCE CRITERIA

- Unit test: 3 monthly same-product charges of ~\$20 plus a disputed \$20 -> R7 fires, verdict legitimate, actions CREATE_CASE, VERIFY_WITH_CUSTOMER, WARN_CUSTOMER, no BLOCK.
- Also consider: low-signal risk_score cases (p <= 0.15 after guards) should close as legitimate, not uncertain.

[HIGH] H4. Patterns not anchored to the flagged transaction; look-ahead

fraud_agent/src/data_loader.py:44; agent.py:44

WHAT IS WRONG

- card_window returns +/- 60 days, including txns after the alert.
- Detectors return any match in the 120-day span, even if the flagged txn is not part of it.
- account_takeover(window[-10:]) uses the last 10 rows of the window, which may be up to 60 days after the alert.

WHY IT MATTERS

An old card-testing burst can mark an unrelated purchase as fraud.

FIX

```
before = [r for r in window if r['ts'] <= flag['ts']]
after  = [r for r in window if r['ts'] > flag['ts']]    # only for 'home activity continues' checks
hit, conf, ids = card_testing(before + [flag])
if flag['TransactionID'] not in ids:
    hit = False
# account_takeover: use the 10 txns ending at the flagged txn
idx = index_of(flag, before_and_flag); ato_rows = before_and_flag[max(0, idx-9): idx+1]
```

ACCEPTANCE CRITERIA

- Every detector hit includes the flagged TransactionID, or is ignored.
- No detector reads rows with ts > flagged ts, except out_of_region's 'home activity continues' check.

[HIGH] H5. Card-testing counts a large first purchase as a test

fraud_agent/src/pattern_detectors.py:13

WHAT IS WRONG

- `small = [on[i]]` adds the starting txn without checking it is under \$5, so one large + two tiny auths count as three tests.

WHY IT MATTERS

R5 needs three or more small online authorizations within an hour.

VERIFIED

Verified: \$900, \$1, \$1, then \$50 within 2h -> (True, 0.85).

FIX

```
for i in range(len(on)):
    if float(on[i]['TransactionAmt']) >= p['R5_card_testing']['small_lt']:
        continue
    small = [on[i]]
    ...
```

ACCEPTANCE CRITERIA

- Test with \$900, \$1, \$1, \$50 returns hit=False.
- Test with \$1, \$2, \$1, \$80 in 60 min returns hit=True.

[HIGH] H6. Exposure, case and SAR disagree

fraud_agent/src/agent.py:129, 147-160

WHAT IS WRONG

- exposure is summed over pattern txns for any verdict, but case.exposure_usd is 0 unless verdict == fraud.
- HHG-010: verdict uncertain, case exposure 0, but SAR file=true with reason "exposure>1000". HHG-019 similar.

WHY IT MATTERS

Policy 3a: a SAR requires fraud confirmed or strongly suspected. Contradictory fields lose points.

FIX

```
exposure = compute_exposure(affected_ids) if verdict == "fraud" else 0.0
# use this single value for case.exposure_usd, sar.total_amount_usd,
# BLOCK_CARD route (L1 <= 2500 < L2) and SAR thresholds
```

ACCEPTANCE CRITERIA

- Validator (see Section 5) passes on all 20 outputs: sar.file implies verdict == fraud (or p >= strong-suspicion threshold from YAML) and sar.total_amount_usd == case.exposure_usd.

[HIGH] H7. 'undocumented' pattern never emitted; SAR over-filed

fraud_agent/src/agent.py:130, 153

WHAT IS WRONG

- undoc = pattern == 'none' and verdict == 'fraud'. So any fraud with no known pattern is treated as R9, and a SAR is filed (HHG-009: \$30.02, SAR filed).
- Yet the answer still says pattern 'none' with an empty pattern_description.
- R9 requires coordinated or repeated abuse across customers, and actions CREATE_CASE, FILE_REPORT, ESCALATE_TO_ANALYST.

WHY IT MATTERS

README: finding and describing undocumented patterns is scored.

FIX

```
if not known_hit and shared_origin_with_other_customers_fraud(ctx):
    pattern = "undocumented"
    pattern_description = (f"{n} customers used device profile '{prof}' "
                          f"within {days} days; amounts {lo}-{hi}; ...")
    actions += CREATE_CASE, FILE_REPORT(L2), ESCALATE_TO_ANALYST
```

ACCEPTANCE CRITERIA

- pattern == 'undocumented' implies a non-empty pattern_description.
- pattern 'none' + verdict 'fraud' alone no longer triggers a SAR.

[HIGH] H8. policy.yaml is mostly ignored

fraud_agent/src/policy_engine.py:12 and others

WHAT IS WRONG

- README says YAML is the only source of truth, but only rules.stop is read.
- Hardcoded: 2500 (L1/L2), 1000 (SAR), 500 (R4/R8), 0.70 (R1), 5.0 and 60 min (R5), 0.85/0.15, 0.10 (R7), 3 days (trip).

WHY IT MATTERS

Changing the YAML changes nothing; breaks the auditability claim.

FIX

```
# policy.yaml additions
rules:
  block_route: {l1_max: 2500.0}
  R7_disputed_recurring: {min_same: 3, amount_tol: 0.10, gap_days: [25, 35]}
  trip_guard: {min_days: 3}
# code
lim = self.p["rules"]["block_route"]["l1_max"]
```

ACCEPTANCE CRITERIA

- grep for numeric literals 2500, 1000, 500, 0.70, 5.0 in fraud_agent/src returns nothing outside tests.

[HIGH] H9. R4, R5, R8 actions do not match the policy

fraud_agent/src/policy_engine.py:16; agent.py:121-126

WHAT IS WRONG

- R5 requires DECLINE_TRANSACTION + STEP_UP_AUTH, and BLOCK_CARD if a purchase over \$100 has already cleared. Code returns DECLINE + VERIFY, never BLOCK.
- R4 (no reply): MONITOR_CARD + DECLINE_TRANSACTION; escalate only if exposure > \$500. Code always adds ESCALATE_TO_ANALYST.
- R8: escalate if uncertain AND (exposure > \$500 OR evidence conflicts).

FIX

```
RULES = {
  "R5": lambda ctx: (["DECLINE_TRANSACTION", "STEP_UP_AUTH"]
                    + (["BLOCK_CARD"] if ctx.cleared_after_test > 100 else [])),
  "R4": lambda ctx: (["MONITOR_CARD", "DECLINE_TRANSACTION"]
                    + (["ESCALATE_TO_ANALYST"] if ctx.exposure > 500 else [])),
}
```

ACCEPTANCE CRITERIA

- One unit test per rule R1 to R10 asserting exact action lists and routes.

[MEDIUM] M10. Dead modules; one would crash

fraud_agent/src/next_best_action.py:24

WHAT IS WRONG

- next_best_action.recommend() calls engine.decide(), which does not exist on PolicyEngine.
- risk_scorer.py, case_manager.py, evidence_collector.py, explainer.py are never imported. PATTERN_LABELS in agent.py is unused.
- agent.py lines 112 and 115 check `deny` inside a branch where deny is always False (dead code).

WHY IT MATTERS

fraud_agent/README.md lists these as working parts.

FIX

Delete unused modules or wire them in with tests. Update fraud_agent/README.md layout accordingly.

ACCEPTANCE CRITERIA

- python -m pyflakes / ruff reports no unused imports or undefined attributes.

[MEDIUM] M11. TigerGraph queries do not match the schema

fraud_agent/src/tigergraph/queries.gsql

WHAT IS WRONG

- Queries use vertex/edge names Account, MadeTxn, UsedDevice, Device.
- schema.gsql defines Customer, Card, Txn, DeviceProfile, OWNS, MADE, FROM_DEVICE, BILLED_IN. They would not install.
- Answers say source 'graph' and ref 'query:...', but data comes from CSV.

WHY IT MATTERS

This is a TigerGraph challenge; graph evidence that is really CSV is a credibility risk.

FIX

```
CREATE QUERY card_window(String card_id, DATETIME t, INT days) FOR GRAPH Fraud {
  start = {Card.*};
  txns = SELECT x FROM start:c -(MADE>)- Txn:x
    WHERE c.id == card_id
    AND abs(datetime_diff(x.ts, t)) <= days*86400;
  PRINT txns;
}
```

ACCEPTANCE CRITERIA

- All queries install on the schema. A GraphStore class exposes the same methods as the CSV Store and can be swapped by env var.
- Until wired, set evidence source to 'csv' honestly.

[MEDIUM] M12. tool_calls accumulates across cases

fraud_agent/src/agent.py:163

WHAT IS WRONG

- One Store instance is shared by all cases and `calls` never resets (outputs show 3, 6 ... 60).

FIX

```
store.calls = 0 # first line of investigate()
```

ACCEPTANCE CRITERIA

- Every answer's tool_calls reflects that case only.

[MEDIUM] M13. Prior-case retrieval is not case-specific

fraud_agent/src/retrieval.py:9

WHAT IS WRONG

- Score = same pattern (+2) + confirmed_fraud (+1); ties by case ID. Every CNP case gets CC-5564, CC-5556, CC-5541.
- The CSV is re-read on every call; `flagged\_notes` is ignored.

FIX

```
score = (2*(r.pattern == pattern) + (r.channel == ch) + (r.region == region)
        + amount_closeness(r.exposure_usd, amt) + (r.card_id in connected_cards))
# load closed_cases_history.csv once at module import
```

ACCEPTANCE CRITERIA

- Two cases with the same pattern but different amount/region get different top-3 lists.

[MEDIUM] M14. Performance on the real 590,742-row file

fraud_agent/src/agent.py:35; data_loader.py:device_neighbors

WHAT IS WRONG

- agent.py copies the full identity dict per case.
- device_neighbors rebuilds a profile string for every identity row, per case.
- Uses the csv module although pandas and pyarrow are already required.

FIX

```
# once, in Store.load()
self.by_profile = defaultdict(set)
for tid, i in self._ident.items():
    self.by_profile[profile(i)].add(self._txn[tid]["customer_id"])
```

ACCEPTANCE CRITERIA

- All 20 cases run in well under a minute after load.

[LOW] L15. Test coverage

fraud_agent/tests/

WHAT IS WRONG

- Only 4 tests, all on PolicyEngine. No golden tests on the 20 outputs.

FIX

```
@pytest.mark.parametrize("rows, expected", CARD_TESTING_CASES)
def test_card_testing(rows, expected):
    assert card_testing(rows)[0] is expected
```

ACCEPTANCE CRITERIA

- tests/test_detectors.py, tests/test_guards.py, tests/test_policy_rules.py, tests/test_answer_validator.py exist and pass.

[LOW] L16. Code readability

fraud_agent/src/agent.py, pattern_detectors.py

WHAT IS WRONG

- Names like ct, cc, ci, cb, bc, bi; lines over 250 chars; `__import__`('collections'); imports inside functions; `open()` without close.

FIX

Introduce the Signal dataclass (Section 5), run `ruff + black`, max line length 100.

[CRITICAL] F1. Face pipeline never checks that the face matches

`src/pipeline.py:166; src/social_filter.py:167`

WHAT IS WRONG

- `pipeline.py` takes `social_matches[0]` (first Bing result) and records it on-chain as the match.
- `FaceEngine.compare_embeddings()` exists (`face_engine.py:204`) but is never called.
- `calculate_confidence()` = 0.60 base + 0.20 if URL is live + 0.15 if it is a post + 0.05 if it has a title. A live post of a stranger scores 0.95 (capped 0.99).

WHY IT MATTERS

The contract permanently certifies an unchecked claim, which defeats a tamper-evident record.

FIX

```
for m in social_matches:
    img = download_image(m.image_url)
    faces = face_engine.process_image(img).faces
    if not faces: continue
    sim = face_engine.compare_embeddings(src_face.embedding, faces[0].embedding)
    if sim >= 0.363:          # SFace cosine threshold
        m.confidence_score = sim
        verified.append(m)
if not verified: abort("no face-verified match; nothing written on-chain")
```

ACCEPTANCE CRITERIA

- No record is written unless cosine similarity $\geq$ threshold.
- `match_confidence` stored on-chain is the similarity, not URL heuristics.

[HIGH] F2. Privacy and legal risk of the face pipeline

`contracts/FaceVerificationRegistry.sol:56; README.md`

WHAT IS WRONG

- Photo in, a person's social profiles out: this can de-anonymise people.
- A face-embedding hash plus the social URL are written to a public chain and cannot be erased.
- India's Digital Personal Data Protection Act 2023 gives a right to erasure of personal data; data on a public chain cannot be erased. `sample_images/` also includes a real public figure's photo.
- Not legal advice; a lawyer should review before any real use.

FIX

Add an explicit consent gate. Keep personal data off-chain. Store only a salted commitment hash on-chain; never the URL or post ID.

ACCEPTANCE CRITERIA

- Contract stores only bytes32 commitment + timestamp + attester.

[MEDIUM] F3. Silent fallback to an in-memory fake chain

`src/blockchain_service.py:124`

WHAT IS WRONG

- With no RPC URL and no local node, it uses `EthereumTesterProvider()`, which disappears when the process exits, but output still says the record was anchored.

FIX

```
if not self.rpc_url and not dev_mode:
    raise SystemExit("No chain configured. Pass --dev for a throwaway in-memory chain.")
```

ACCEPTANCE CRITERIA

- Running without RPC and without `--dev` exits with a clear error.

[MEDIUM] F4. Reverse search depends on scraping bing.com

`scripts/reverse_search.js:12, 41`

WHAT IS WRONG

- Puppeteer drives `bing.com` and parses HTML; breaks on any page change and may violate the site's terms.
- Chrome path hardcoded to `/usr/bin/google-chrome-stable`.

FIX

Make the search engine a pluggable interface; prefer an official API (for example Google Cloud Vision web detection). Read Chrome path from `env`.

[LOW] F5. Errors are swallowed

`src/blockchain_service.py:118, 165, 209`

WHAT IS WRONG

- Several ``except Exception: pass`` blocks hide real failures.

FIX

Catch specific exceptions and log them.

[HIGH] R1. Two projects in one repo

`README.md`

WHAT IS WRONG

- Root README only documents the face pipeline; the fraud agent is hidden in a subfolder. Repo name typo: `'Invsestigator'`.

FIX

Split into two repos, or a monorepo with `/face-verify` and `/fraud-agent`, each with its own README and requirements.

[MEDIUM] R2. Results not reproducible

`.gitignore`

WHAT IS WRONG

- `transactions.csv` and `identity.csv` are ignored (correct, they are large) but there is no fetch script or checksum.
- 38 MB of ONNX models committed directly to git.

FIX

Add `scripts/get_data.sh` with sha256 checks. Move models to Git LFS or a download step.

[LOW] R3. Hygiene

```
fraud_agent/.env.example:3; requirements
```

WHAT IS WRONG

- Default username typo 'tigergrpah'. No CI. Requirements use >= so builds drift. No fraud_agent/__init__.py.

FIX

Add GitHub Actions (ruff + pytest on push). Pin versions.

5. Target architecture for the fraud agent

Split the single 150-line investigate() into small pure steps. Each step takes plain data and returns plain data, so each can be tested alone.

```
Loader          build indexes once (txn by id, txns by customer, customers by device profile)
-> Features      split window into before/after the flagged txn; medians per product
-> Detectors      one module per pattern, each returns Signal objects
-> Guards         trip guard, R7: REMOVE signals (never just relabel)
-> Scorer         single calibrated probability from remaining signals + bank score
-> Policy         R1..R10 as data-driven rules reading policy.yaml
-> Answer         build JSON with ONE exposure value
-> Validator      cross-field checks; fail loudly
-> Writer
```

Proposed layout

```
fraud_agent/
  policies/policy.yaml          # every threshold lives here
  src/
    io/store.py                 # CSV store + indexes
    io/graph_store.py           # TigerGraph store, same interface
    features.py
    detectors/
      base.py                   # Signal dataclass
      card_testing.py cnp.py    cnp_new_device.py out_of_region.py account_takeover.py
      undocumented.py           # cross-customer shared-origin detection (R9)
    guards.py                   # trip guard, R7
    scoring.py
    policy.py                   # R1..R10
    answer.py                   # build + validate
    agent.py                    # thin orchestration + CLI
  tests/
    test_detectors.py test_guards.py test_policy_rules.py
    test_answer_validator.py test_golden_cases.py
```

Shared data type

```
from dataclasses import dataclass

@dataclass(frozen=True)
class Signal:
    name: str                    # "card_testing", "card_not_present_fraud", ...
    hit: bool
    conf: float                  # 0..1
    txn_ids: tuple[str, ...]     # must include the flagged txn id
    why: str                     # plain-English reason, used in evidence claims

signals = [s for s in detect_all(ctx) if s.hit and ctx.flag_id in s.txn_ids]
signals = apply_guards(signals, ctx)          # guards drop signals
p = score(signals, ctx.bank_score)
```

Answer validator checks (run on every answer before writing)

- sar.file == true implies verdict == 'fraud' (or p >= YAML strong-suspicion threshold) and at least one 3a condition holds.
- sar.total_amount_usd == case.exposure_usd; exposure_usd == 0 when verdict != 'fraud'.
- pattern == 'undocumented' implies non-empty pattern_description.
- pattern is one of the six allowed values; verdict and status consistent (fraud -> closed_fraud, etc.).
- FILE_REPORT present in final actions if and only if sar.file is true; FILE_REPORT route is L2.
- BLOCK_CARD route matches exposure (L1 if <= 2500, else L2). BLOCK_ALL_CARDS only if R10 holds.
- Every ID in the answer exists in the dataset (README rule).
- No BLOCK_* action when R1 applies (single signal and p < 0.70) before verification.

6. Fix order (do in this sequence)

#	Task	Findings	Done when
1	Add synthetic-data tests for detectors, then fix the three detector bugs	C1, H5, H4	New detector tests pass
2	Rebuild signals after guards	C2	Guard tests pass; HHG-001 p no longer from dropped signal
3	Implement real R7 (same product, monthly, per-product median)	C3	R7 test passes
4	Single exposure value + answer validator	H6, H7	Validator passes on all 20 regenerated answers
5	Move every threshold into policy.yaml; one test per rule R1 to R10	H8, H9	No numeric literals in src; rule tests pass
6	Emit 'undocumented' with description from cross-customer evidence	H7	At least the shared-device case is labelled correctly
7	Golden tests on the 20 regenerated outputs	L15	Snapshot test in CI
8	Delete dead modules; reset tool_calls; index device profiles; retrieval scoring	M10, M12, M13, M14	ruff clean; run time acceptable
9	Fix GSQL names, install queries, add GraphStore	M11	Queries install; env var swaps store
10	Split repo / restructure READMEs; data fetch script; LFS for models; CI	R1, R2, R3	Fresh clone reproduces outputs with one script
11	Face pipeline: face verification, consent, off-chain data, no silent fake chain	F1 to F5	Only if the project continues

7. How this review was done, and limits

- Cloned the repo and read every Python, GSQL, YAML and Solidity file.
- Read all 20 answer files in fraud_agent/cases/ and tabulated verdicts, patterns, exposure, SAR flags, tool_calls.
- Reproduced C1, C2 and H5 with small synthetic inputs against the committed functions.
- Could not run the full agent: transactions.csv (about 708 MB) is not in the repo. Real accuracy was not measured; the answer key is private.
- After fixes, the implementing model should regenerate all 20 answers and compare before/after verdict counts. Expect a meaningful share of legitimate verdicts.